

SIMATS ENGINEERING ASSESSMENT TOOLS

COURSE NAME: Theory of Computation COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO4: Construct Context-Free Grammars, generate derivations and parse trees to demonstrate the syntactic structure of strings. (BL : 3)

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:100 Duration : 1 Hour

Industry Problem

Code of Conduct:

I Pusalapati Chandu(192372119) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



SIMATS ENGINEERING ASSESSMENT TOOLS

1.Design and implement a **C-based validation module** that simulates a **Deterministic Finite Automaton (DFA)** to verify input strings in a real-time system. The module should accept only those strings that **begin with the character 'a' and end with the character 'a'**, ensuring compliance with predefined input format rules (e.g., protocol validation, pattern filtering, or secure data transmission).

The system should:

- Process user input dynamically
- Validate strings based on DFA state transitions
- Output whether the given input is **Accepted** or **Rejected**

2. Design a Push Down Automata that accepts the language

$$L = \{w \mid w \in (0 + 1)^* \text{ and } n_0(w) = n_1(w)\}$$

$n_0(w)$ is the number of 0's in w

$n_1(w)$ is the number of 1's in w

3. Design a Pushdown Automata for the language representing a's followed by b's and the number of b's must be twice than that of a's.

$$L = \{a^n b^{2n} \mid n \geq 1\}$$

4. Develop a **C-based computational module** to determine the **ϵ -closure of all states** in a **Non Deterministic Finite Automaton (NFA) with ϵ -transitions**, as part of an automata processing engine used in compiler construction and pattern-matching systems.

5. Develop a **C-based input validation module** that determines whether a given binary string conforms to a specified **Context-Free Grammar (CFG)** used in structured data validation systems.

The grammar is defined as:

- $S \rightarrow 0 A 1$
- $A \rightarrow 0 A \mid 1 A \mid \epsilon$

SIMATS ENGINEERING ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0– 1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

QUESTION 1 – C-BASED DFA VALIDATION MODULE

The system must accept only strings that begin with 'a' and end with 'a'. It must process user input dynamically, use DFA state transitions and print Accepted or Rejected. □filecite□turn4file0□L24-L28□

Aim

To design and implement a deterministic finite automaton in C that accepts a string only when its first and last characters are both 'a'.

Language

$L = \{ w \in \{a,b\}^* \mid w \text{ starts with } a \text{ and ends with } a \}$

DFA States

State	Meaning	On a	On b
q0	Start	q1	qReject
q1	Current last symbol is a	q1	q2
q2	Current last symbol is b	q1	q2
qReject	Invalid first character	qReject	qReject

Algorithm

1. Start in q0.
2. Read the input one character at a time.
3. If the first character is a, move to q1; otherwise move to qReject.
4. From q1, a keeps the DFA in q1 and b moves to q2.
5. From q2, a moves to q1 and b stays in q2.
6. After the whole input is read, accept only when the state is q1.

C Code

```
#include <stdio.h>

int main() {
    char str[100];
    int state = 0;

    printf("Enter a string: ");
    scanf("%99s", str);

    for (int i = 0; str[i] != '\0'; i++) {
        char ch = str[i];

        if (state == 0) {
            if (ch == 'a')
                state = 1;
        }
    }
}
```

```

        else
            state = 3;
    }
    else if (state == 1) {
        if (ch == 'a')
            state = 1;
        else if (ch == 'b')
            state = 2;
        else
            state = 3;
    }
    else if (state == 2) {
        if (ch == 'a')
            state = 1;
        else if (ch == 'b')
            state = 2;
        else
            state = 3;
    }
}

if (state == 1)
    printf("Accepted\n");
else
    printf("Rejected\n");

return 0;
}

```

Input and Output

Input:

Enter a string: abba

Output:

Accepted

Input:

Enter a string: abb

Output:

Rejected

Justification

- abba begins with a and ends with a, so it is accepted.
- abb begins with a but ends with b, so it is rejected.

- baa is rejected because the first character is not a.
- The DFA is deterministic because each state/input combination has one next state.

Conclusion

The DFA validates the required pattern in one left-to-right scan. Its time complexity is $O(n)$, where n is the input length.

QUESTION 2 – PUSH DOWN AUTOMATA

The paper asks to design a PDA for a language involving the number of 0s and the number of 1s, but the exact mathematical relationship is missing from the extracted text. Only the statements referring to the number of 0s and 1s are visible. □filecite□turn4file0□L29-L33□

Important Note

An exact PDA cannot be safely written without the missing mathematical condition. The following gives the standard PDA method for the common equal-count interpretation, without claiming that the omitted formula is definitely that language.

If the intended language is equal numbers of 0s and 1s

The stack can be used to match one symbol against another. For an ordered language such as 0^n1^n , push a marker for every 0 and pop one marker for every 1.

Algorithm

7. Start with a bottom stack marker Z_0 .
8. For each 0, push X onto the stack.
9. When 1s begin, pop one X for each 1.
10. Do not allow 0 after entering the 1-processing state for the ordered form 0^n1^n .
11. Accept when all input is consumed and only Z_0 remains.

Example for 0011

```
Read 0 -> push X
Read 0 -> push X
Read 1 -> pop X
Read 1 -> pop X
End -> only  $Z_0$  remains
Result -> Accepted
```

Justification

A PDA is suitable because the stack provides unbounded memory. A finite automaton cannot generally remember an arbitrary number of 0s, while a PDA can store one marker for each symbol and later remove those markers.

Conclusion

The exact answer for Q2 depends on the mathematical condition omitted from the uploaded document. The method above should be replaced with the exact PDA once that missing expression is visible.

QUESTION 3 – PDA FOR $L = \{a^n b^{2n} \mid n \geq 1\}$

The problem explicitly asks for a PDA accepting a's followed by b's, with the number of b's twice the number of a's: $L = \{a^n b^{2n} \mid n \geq 1\}$. □ filecite□turn4file0□L33-L34□

Aim

To design a PDA that accepts strings of the form $a^n b^{2n}$, where every a requires exactly two b's.

Main Idea

- For every a, push two X markers.
- After the first b is read, move to the b-processing state.
- For every b, pop one X.
- If an a appears after b processing begins, reject.
- Accept only when the input ends exactly as all X markers are removed.

PDA Components

Component	Definition
States	q_0 = read a's, q_1 = read b's, q_{Accept}
Input alphabet	$\{a, b\}$
Stack alphabet	$\{Z_0, X\}$
Initial symbol	Z_0
Acceptance	Input consumed and stack returns to Z_0

Transition Logic

State	Input	Stack	Action	Next
q_0	a	Z_0	Push XX	q_0
q_0	a	X	Push XX	q_0
q_0	b	X	Pop X	q_1
q_1	b	X	Pop X	q_1
q_1	a	X	Reject	—
q_1	end	Z_0	Accept	q_{Accept}

Examples

String	Required b count	Result
abb	2	Accepted

aabbbb	4	Accepted
aaabbbbb	6	Accepted
aabb	4 required	Rejected
abbb	2 required	Rejected

Explanation

For aabbbb, two a's are read. The PDA pushes two X symbols for each a, so four X symbols are stored. The four b's pop the four X symbols exactly. Therefore the string is accepted.

Conclusion

The stack acts as a counter storing $2n$ markers. Each b removes one marker, so exactly $2n$ b's are required.

QUESTION 4 – ϵ -CLOSURE OF AN NFA

The problem asks for a C-based module to determine the ϵ -closure of all states in an NFA with ϵ -transitions, for use in compiler construction and pattern matching. □filecite□turn4file0□L35-L35□

Aim

To calculate the ϵ -closure of every state of an NFA using depth-first traversal.

Definition

The ϵ -closure of a state is the set containing that state and every state reachable from it using zero or more ϵ -transitions.

Algorithm

12. Read the number of states.
13. Read the ϵ -transition matrix.
14. For each state, include the state itself in its closure.
15. Perform DFS from that state.
16. Whenever an ϵ -transition reaches an unvisited state, add it and continue DFS.
17. After no new states are found, print the closure.

C Code

```
#include <stdio.h>

#define MAX 20

int eps[MAX][MAX];
int closure[MAX][MAX];
int n;

void dfs(int start, int state) {
    for (int next = 0; next < n; next++) {
        if (eps[state][next] && !closure[start][next]) {
            closure[start][next] = 1;
            dfs(start, next);
        }
    }
}
```



```

    }
}

int main() {
    printf("Enter number of states: ");
    scanf("%d", &n);

    printf("Enter epsilon transition matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &eps[i][j]);

    for (int i = 0; i < n; i++) {
        closure[i][i] = 1;
        dfs(i, i);
    }

    printf("\nEpsilon Closures:\n");
    for (int i = 0; i < n; i++) {
        printf("E-closure(q%d) = { ", i);
        for (int j = 0; j < n; j++)
            if (closure[i][j])
                printf("q%d ", j);
        printf("}\n");
    }

    return 0;
}

```

Sample Input

```

Enter number of states: 4

Enter epsilon transition matrix:
1 1 0 0
0 1 1 0
0 0 1 1
0 0 0 1

```

Sample Output

```

Epsilon Closures:
E-closure(q0) = { q0 q1 q2 q3 }
E-closure(q1) = { q1 q2 q3 }
E-closure(q2) = { q2 q3 }
E-closure(q3) = { q3 }

```

Explanation

- q_0 reaches q_1 through ϵ .
- q_1 reaches q_2 through ϵ .
- q_2 reaches q_3 through ϵ .
- Therefore q_0 reaches all four states.
- Every state belongs to its own ϵ -closure because zero transitions are allowed.

Conclusion

The DFS-based module finds all states reachable through ϵ -transitions and can be used as part of NFA-to-DFA conversion and pattern-matching engines.

QUESTION 5 – CFG INPUT VALIDATION

The problem asks for a C-based module that validates a binary string against the CFG $S \rightarrow 0A1$, $A \rightarrow 0A \mid 1A \mid \epsilon$.

Aim

To determine whether a user-given binary string can be generated by the specified CFG.

Grammar

```
S -> 0 A 1
A -> 0 A
A -> 1 A
A -> ε
```

Deriving the Language

S always contributes a 0 at the beginning and a 1 at the end. A can generate any sequence of 0s and 1s, including the empty string. Therefore:

$$L = \{ 0w1 \mid w \in \{0,1\}^* \}$$

In simple words, the input must be a binary string that starts with 0 and ends with 1.

Examples

Input	Reason	Result
01	$A \rightarrow \epsilon$	Accepted
001	$A \rightarrow 0$	Accepted
011	$A \rightarrow 1$	Accepted
0101	$A \rightarrow 01$	Accepted
10	Wrong first/last symbols	Rejected
00	Does not end in 1	Rejected
11	Does not start in 0	Rejected

C Code

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int len, valid = 1;

    printf("Enter a binary string: ");
    scanf("%99s", str);

    len = strlen(str);

    if (len < 2)
        valid = 0;

    for (int i = 0; i < len; i++) {
        if (str[i] != '0' && str[i] != '1') {
            valid = 0;
            break;
        }
    }

    if (valid && str[0] == '0' && str[len - 1] == '1')
        printf("Accepted by the CFG\n");
    else
        printf("Rejected by the CFG\n");

    return 0;
}
```

Sample Input and Output

Input:
Enter a binary string: 0101

Output:
Accepted by the CFG

Input:
Enter a binary string: 1010

Output:
Rejected by the CFG

Derivation Example

$S \Rightarrow 0A1$
 $\Rightarrow 0(0A)1$
 $\Rightarrow 0(01)1$
 $\Rightarrow 0101$

The middle part 01 is generated by A using $A \rightarrow 0A$, then $A \rightarrow 1A$, and finally $A \rightarrow \epsilon$.

Justification

- The first character must be 0 because of $S \rightarrow 0A1$.
- The last character must be 1 because of $S \rightarrow 0A1$.
- The middle can contain any binary sequence because A can repeatedly generate 0 or 1.
- Any non-binary input is rejected by the validation program.

Conclusion

The CFG generates exactly the binary strings that start with 0 and end with 1. The C module validates this condition directly.

FINAL ANSWER SUMMARY

Q	Topic	Main Result
1	DFA	Accept strings beginning with a and ending with a.
2	PDA	Exact mathematical condition is missing from the uploaded text; standard count-based PDA method explained.
3	PDA $a^n b^{2n}$	Push two markers for every a and pop one for every b.
4	ϵ -closure	Use DFS to find all states reachable through ϵ -transitions.
5	CFG	Accept binary strings beginning with 0 and ending with 1.

IMPORTANT NOTE

Question 2 cannot be completed as an exact final PDA from the extracted document because the mathematical language condition is missing. The document deliberately identifies this limitation rather than inventing a condition. □ filecite□turn4file0□L29-L33□

CONCLUSION

The five industry problems apply DFA, PDA, ϵ -closure and CFG concepts to practical validation and processing systems. The solutions emphasize state transitions, stack operations, derivations and systematic validation.